

串行通信与并行通信

并行：例如使用8根数据线一次传送1个字节，使用16根数据线一次传送2个字节

串行：使用少量数据信号线将数据逐位分时传送

串行通信 { 通信距离：远
抗干扰能力：强
传输速率：低
成本：低

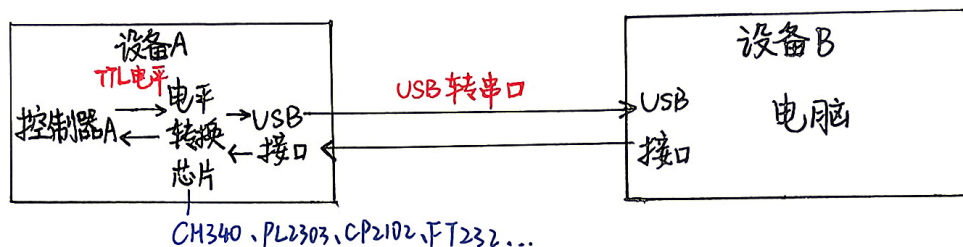
并行通信 { 通信距离：近
抗干扰能力：弱
传输速率：高
成本：高

在工作频率相同的情况下，并行通信的数据传输速率明显高于串行通信

由于并行传输对同步要求较高，且随通信速率提高，信号干扰问题显著影响通信性能

高速率串行差分传输成为越来越多场合中的应用

USB 转串口



全双工通信 (同时双向)

站A $\longleftrightarrow$ 站B

半双工通信 (分时双向)

站A $\longleftrightarrow$ 站B

单工通信 (单向)

站A $\longrightarrow$ 站B

同步通信 数据信号、时钟信号同步传输。如 SPI、I2C 通信接口

异步通信 数据信号 (无同步时钟信号)。如 UART、I2wire 单总线

Universal Asynchronous Receiver/Transmitter

同步通信中，数据信号传输内容绝大部分为有效数据，而异步通信中，有帧的各种标识符

同步通信效率更高，但双方时钟的允许误差更小

异步串口通信的串口数据包的基本组成：(起止式)

① 起始位：由1个逻辑0的数据位表示

② 结束位：由0.5、1、1.5、2个逻辑1的数据位表示

③ 有效数据：紧跟在起始位后，其长度常被约定为5、6、7、8位长
(ASCII码至少要7位，扩展码要8位)
0~9 5位就行

有效数据位 + 校验位

传送数据总位数

④ 校验位：(使数据抗干扰)

奇校验 odd、偶校验 even、

0 校验 space、1 校验 mark、无校验 parity

校验位总为0 校验位总为1 数据包中不包含校验位

异步通信协议举例: 9600 - 8 - ^无N - 1 - ^无N - ^{停止位(S)}
bps 位秒(B) | 奇偶校验(P) 数据位(D) 流控制(F)

RTS, CTS 这样的握手信号控制串行传输能否进行。
可通过控制它们的波形来控制数据传输速度(密集就快)

若按照这个协议发送字符 'A' (ASCII 码为 01000001B), 将 8 位数据按 先低后高原则 装配上
起始位、可能的校验位、停止位后得数据帧: 0 1000001 0 1

校验位(可选)

★ 异步串行通信

- 嵌入式开发板不具备自主开发能力
- 使硬件资源消耗减少, 实现原理减少, 提供了简单易行的连接开发板和上位 PC 机的方式
- 可连接 PC 机串口, 用于程序下载
- 借 PC 端的串口调试工具可实现开发板调试、输出
- 开发板可通过串口接收上位机命令并作出响应

STM32 有 3 个 USART + 2 个 UART
同 ' 异 ' 异

USART1 连接高速 APB2 总线、运行速度 72MHz (支持 4.5Mbps 的传输速度)

USART2、USART3、UART4、UART5 位于 APB1 总线、36MHz (2.25Mbps 传输速率)
直接存储器访问, 解决存储器与外设间高速传输

每个 USART 接口有 2 个 DMA 通道

用于接收 Rx 和发送 Tx 数据与存储器之间高速传送

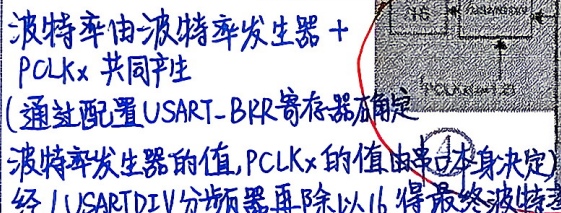
有多种操作模式: 异步全双工通信、同步单路通信、半双工单线通信

$$USARTDIV = \frac{72000000}{16 \times 115200} = 39.0625$$

小数部分 $\text{DIV-Fraction} = 16 \times 0.0625 = 1 = 0 \times 0 \rightarrow \text{bit}[3:0]$

故设置 USART1 → BRR = 0x0271 即可实现。

图248 USART框图



发送数据过程:

由CPU写数据到发送数据寄存器TDR,再由TDR一次性将要发送的数据写入发送移位寄存器,按波特率逐位移出。

接收数据过程:

当包含数据的一帧发送完成且TXE发送数据寄存器空时, TC位(发送完成置1, 就可以
来移位寄存器接收移位寄存器, 1帧接收完以后, 移位寄存器给下一个数据

由Rx接收数据,按波特率按位写入接收移位寄存器,待接收完毕后一次性写入接收数据寄存器RDR, CPU读取寄存器,获取传输的数据

STM32F103 共5个串口

串口时钟来自PLCK2 (APB2外设的时钟, 72MHz)

串口2~5时钟来自PLCK1 (APB1 外设的时钟, 36MHz)

例如: 串口的时钟由PLCK2 经 /USARTDIV (分频) 得到, 而分频由波特率发生器控制, 通过相关寄存器配置

USART 接口通过

- { Tx: 发送数据输出
- { Rx: 接收数据输入
- { CK: 发送时钟输出, 用于同步传输模式 (SCLK)

这3个引脚连接外设.

实现硬件流程控制需

- { nCTS: 清除发送
 - { nRTS: 发送请求
- 这2个引脚

通知发送方“我已准备好接收

“你可以发数据给我”.

当串口准备好接收新数据时, 由硬件自动把 RTS 脚拉低, 向外表示可接收数据; 在发送数据前, 由硬件自动检查 CTS 脚是否为低, 表示是否可发送数据, 再进行发送.

引脚	APB2 总线 USART1	寄存器	
		USART-SR	状态寄存器
TX	PA9	USART_DR	数据寄存器
RX	PA10	USART_BRR	波特率寄存器
SCLK	PA8	USART_CR1	控制寄存器1
nCTS	PA11	USART_CR2	控制寄存器2
nRTS	PA12	USART_CR3	控制寄存器3
		USART_GTPR	时间保护和预分频寄存器

检查指定的 USART 标志位设置与否

FlagStatus USART_GetFlagStatus (USART_TypeDef * USARTx, uint16 USART_FLAG);

USART_FLAG 值

USART_FLAG_CTS	CTS 标志位
USART_FLAG_LBD	LIN 中断检测标志位
USART_FLAG_TXE	发送数据寄存器空标志位
USART_FLAG_TC	发送完成标志位
USART_FLAG_RXNE	接收数据寄存器非空标志位
USART_FLAG_IDLE	空闲总线标志位
USART_FLAG_ORE	溢出错误标志位
USART_FLAG_NE	噪声错误标志位
USART_FLAG_FE	帧错误标志位
USART_FLAG_PE	奇偶错误标志位

例如. FlagStatus Status;

Status = USART_GetFlagStatus (USART1, USART_FLAG_TXE);

清除 USARTx 的特处理标志位

void USART_ClearFlag (USART_TypeDef * USARTx, uint16 USART_FLAG);

例如. USART_ClearFlag (USART1, USART_FLAG_ORE);

中断状态位获取函数

ITStatus USART_GetITStatus (USART_TypeDef * USARTx, uint16 USART_IT);

例如. ITStatus ErrorITStatus;

ErrorITStatus = USART_GetITStatus (USART1, USART_IT_ORE);

清除 USARTx 的中断特处理位

void USART_ClearITPendingBit (USART_TypeDef * USARTx, uint16 USART_IT);

例如. USART_ClearITPendingBit (USART1, USART_IT_ORE);

串口使能函数

```
void USART_Cmd(USART_TypeDef * USARTx, FunctionalState NewState);  
USART_Cmd(USART1, ENABLE);  
                  DISABLE
```

中断配置函数

使能 or 失能 USART 中断, 可取一个/多个值的组合

```
void USART_ITConfig(USART_TypeDef * USARTx, uint16_t USART_IT, FunctionalState NewState);
```

例如. USART_ITConfig(USART1, USART_IT_Transmit ENABLE);

USART_IT 的值

USART_IT-PE	奇偶错误中断
USART_IT-TXE	发送中断
USART_IT-TC	传输完成中断
USART_IT-RXNE	接收中断
USART_IT-IDLE	空闲总线中断
USART_IT-LBD	LIN 中断检测中断
USART_IT-CTS	CTS 中断
USART_IT-ERR	错误中断

数据发送函数

```
void USART_SendData(USART_TypeDef * USARTx, uint16_t Data);
```

例如. USART_SendData(USART3, 0x26);

数据接收函数

```
uint16_t USART_ReceiveData(USART_TypeDef * USARTx);
```

前提: RXNE 接收缓冲寄存器非空时, 收才有意义

例如. uint16_t RxData;
RxData = USART_ReceiveData(USART2);

串口操作相关库函数

void USART_Init(); // 串口初始化: 波特率, 数据字长, 奇偶校验, 硬件流控, 收发使能

void USART_Cmd(); // 使能串口

void USART_ITConfig(); // 使能相关中断

void USART_SendData(); // 发送数据到串口, DR

uint16_t USART_ReceiveData(); // 接收数据, 从DR读取接受到的数据

FlagStatus USART_GetFlagStatus(); // 获取状态标志位

void USART_ClearFlag(); // 清除状态标志位

ITStatus USART_GetITStatus(); // 获取中断状态标志位

void USART_ClearITPendingBit(); // 清除中断状态标志位

串口初始化函数

要配置的串口

指向 USART_InitTypeDef 结构变量的指针

void USART_Init(USART_TypeDef *USARTx, USART_InitTypeDef *USART_InitStructure);

typedef struct

```
{
    uint32_t USART_BaudRate; // 通信波特率
    uint16_t USART_WordLength; // 数据位数
    uint16_t USART_StopBits; // 停止位数
    uint16_t USART_Parity; // 校验模式
    uint16_t USART_Mode; // 接收/发送模式
    uint16_t USART_HardwareFlowControl; // 硬件流控制
    ... 另外4个结构体成员仅用于同步模式, 这里略过
}
```

} USART_InitTypeDef;

例: USART_InitTypeDef USART_InitStructure;

USART_InitStructure.USART_BaudRate = 9600;

USART_InitStructure.USART_WordLength = USART_WordLength_8b;

USART_InitStructure.USART_StopBits = USART_StopBits_1;

USART_InitStructure.USART_Parity = USART_Parity_Odd; // Even / Space / Mark / NoParity

USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_RTS-CTS;

USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;

USART_Init(USART1, &USART_InitStructure);

APB2

解读: 9600-8-奇-1-Y 收发

25.6.1 状态寄存器(USART_SR)

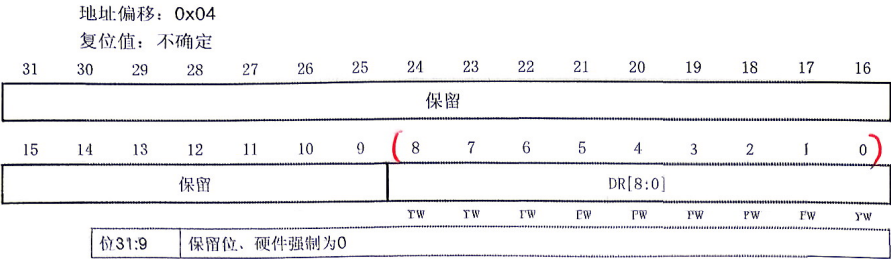
地址偏移: 0x00
复位值: 0x00C0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留						CTS	LBD	TXE	TC	RXNE	LOLE	ORE	NE	FE	PE
						re w0	re w0	r	re w0	re w0	r	g	f	p	f
位31:10	保留位, 硬件强制为0														
位9	CTS: CTS 标志 (CTS flag) 如果设置了CTSE位, 当nCTS输入变化状态时, 该位被硬件置高。由软件将其清零。如果USART_CR3中的CTSIE为'1', 则产生中断。 0: nCTS状态线上没有变化; 1: nCTS状态线上发生变化。 注: UART4和UART5上不存在这一位。														
位8	LBD: LIN断开检测标志 (LIN break detection flag) 当检测到LIN断开时, 该位由硬件置'1', 由软件清'0'(向该位写0)。如果USART_CR3中的LBDIE = 1, 则产生中断。 0: 没有检测到LIN断开; 1: 检测到LIN断开。 注意: 若LBDIE=1, 当LBD为'1'时要产生中断。														
位7	TXE: 发送数据寄存器空 (Transmit data register empty) 当TDR寄存器中的数据被硬件转移到移位寄存器的时候, 该位被硬件置位。如果USART_CR1寄存器中的TXEIE为1, 则产生中断。对USART_DR的写操作, 将该位清零。 0: 数据还没有被转移到移位寄存器; 1: 数据已经被转移到移位寄存器。 注意: 单缓冲器传输中使用该位。														
位6	TC: 发送完成 (Transmission complete) 当包含有数据的一帧发送完成后, 并且TXE=1时, 由硬件将该位置'1'。如果USART_CR1中的TCIE为'1', 则产生中断。由软件序列清除该位(先读USART_SR, 然后写入USART_DR)。TC位也可以通过写入'0'来清除, 只有在多缓存通讯中才推荐这种清除程序。 0: 发送还未完成; 1: 发送完成。														
位5	RXNE: 读数据寄存器非空 (Read data register not empty) 当RDR移位寄存器中的数据被转移到USART_DR寄存器中, 该位被硬件置位。如果USART_CR1寄存器中的RXNEIE为1, 则产生中断。对USART_DR的读操作可以将该位清零。RXNE位也可以通过写入0来清除, 只有在多缓存通讯中才推荐这种清除程序。 0: 数据没有收到; 1: 收到数据, 可以读出。														

通过读取该位的值, 判断是否收到完整数据并写入USART_DR寄存器

位4	IDLE: 监测到总线空闲 (IDLE line detected) 当检测到总线空闲时, 该位被硬件置位。如果USART_CR1中的IDLEIE为'1', 则产生中断。由软件序列清除该位(先读USART_SR, 然后读USART_DR)。 0: 没有检测到空闲总线; 1: 检测到空闲总线。 注意: IDLE位不会再次被置高直到RXNE位被置起(即又检测到一次空闲总线)
位3	ORE: 过载错误 (Overrun error) 当RXNE仍然是'1'的时候, 当前被接收在移位寄存器中的数据, 需要传送至RDR寄存器时, 硬件将该位置位。如果USART_CR1中的RXNEIE为'1'的话, 则产生中断。由软件序列将其清零(先读USART_SR, 然后读USART_CR)。 0: 没有过载错误; 1: 检测到过载错误。 注意: 该位被置位时, RDR寄存器中的值不会丢失, 但是移位寄存器中的数据会被覆盖。如果设置了EIE位, 在多缓冲器通信模式下, ORE标志置位会产生中断的。
位2	NE: 噪声错误标志 (Noise error flag) 在接收到的帧检测到噪音时, 由硬件对该位置位。由软件序列对其清零(先读USART_SR, 再读USART_DR)。 0: 没有检测到噪声; 1: 检测到噪声。 注意: 该位不会产生中断, 因为它和RXNE一起出现, 硬件会在设置RXNE标志时产生中断。在多缓冲区通信模式下, 如果设置了EIE位, 则设置NE标志时会产生中断。
位1	FE: 帧错误 (Framing error) 当检测到同步错位, 过多的噪声或者检测到断开符, 该位被硬件置位。由软件序列将其清零(先读USART_SR, 再读USART_DR)。 0: 没有检测到帧错误; 1: 检测到帧错误或者break符。 注意: 该位不会产生中断, 因为它和RXNE一起出现, 硬件会在设置RXNE标志时产生中断。如果当前传输的数据既产生了帧错误, 又产生了过载错误, 硬件还是会继续该数据的传输, 并且只设置ORE标志位。 在多缓冲区通信模式下, 如果设置了EIE位, 则设置FE标志时会产生中断。
位0	PE: 校验错误 (Parity error) 在接收模式下, 如果出现奇偶校验错误, 硬件对该位置位。由软件序列对其清零(依次读USART_SR和USART_DR)。在清除PE位前, 软件必须等待RXNE标志位被置'1'。如果USART_CR1中的PEIE为'1', 则产生中断。 0: 没有奇偶校验错误; 1: 奇偶校验错误。

25.6.2 数据寄存器(USART_DR)



读: 读取该寄存器接收到的数据
写: 向该寄存器写入发送的数据, 对数据进行发送

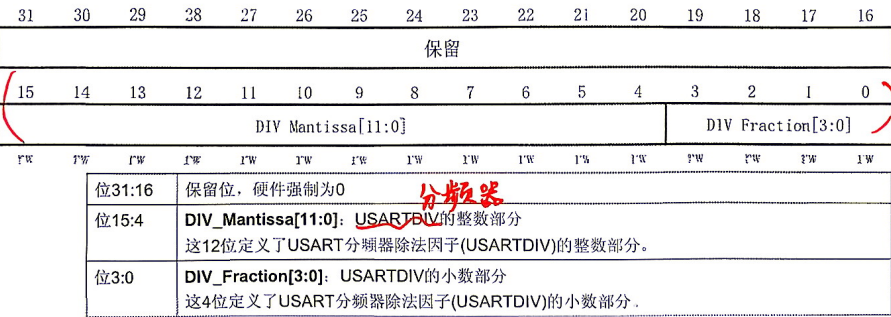
541/754
参照2009年12月 RM0008 Reference Manual 英文第10版
本译文仅供参考, 如有翻译错误, 请以英文原稿为准。请读者随时注意在ST网站上获取新版本

通用同步异步收发器(USART) STM32F10xxx参考手册

位8:0	DR[8:0]: 数据值 (Data value) 包含了发送或接收的数据。由于它是由两个寄存器组成的, 一个给发送用(TDR), 一个给接收用(RDR), 该寄存器兼具读和写的功能。TDR寄存器提供了内部总线和输出移位寄存器之间的并行接口(参见图248)。RDR寄存器提供了输入移位寄存器和内部总线之间的并行接口。 当使能校验位(USART_CR1中PCE位被置位)进行发送时, 写到MSB的值(根据数据的长度不同, MSB是第7位或者第8位)会被后来的校验位该取代。 当使能校验位进行接收时, 读到的MSB位是接收到的校验位。
------	---

25.6.3 波特比率寄存器(USART_BRR)

注意: 如果TE或RE被分别禁止, 波特计数器停止计数
地址偏移: 0x08
复位值: 0x0000



地址偏移: 0x0C
复位值: 0x0000

复位值: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK	
Yes	1W	1W	1W	1W	1W	1W	1W	1W	1W	1W	1W	1W	1W	1W	1W
位31:14	保留位、硬件强制为0。														
位13	UE: USART使能 (USART enable) 当该位被清零, 在当前字节传输完成后USART的分频器和输出停止工作, 以减少功耗。该位由软件设置和清零。 0: USART分频器和输出被禁止; 1: USART模块使能。														
位12	M: 字长 (Word length) 该位定义了数据字的长度, 由软件对其设置和清零 0: 一个起始位, 8个数据位, n个停止位; 1: 一个起始位, 9个数据位, n个停止位。 注意: 在数据传输过程中(发送或者接收时), 不能修改这个位。														
位11	WAKE: 唤醒的方法 (Wakeup method) 这位决定了把USART唤醒的方法, 由软件对该位设置和清零。 0: 被空闲总线唤醒; 1: 被地址标记唤醒。														

位10	PCE: 检验控制使能 (Parity control enable) 用该位选择是否进行硬件校验控制(对于发送来说就是校验位的产生; 对于接收来说就是校验位的检测)。当使能了该位, 在发送数据的最高位(如果M=1, 最高位就是第9位; 如果M=0, 最高位就是第8位)插入校验位; 对接收到的数据检查其校验位。软件对它置'1'或清'0'。一旦设置了该位, 当前字节传输完成后, 校验控制才生效。 0: 禁止校验控制; 1: 使能校验控制。
位9	PS: 校验选择 (Parity selection) 当校验控制使能后, 该位用来选择是采用偶校验还是奇校验。软件对它置'1'或清'0', 当前字节传输完成后, 该选择生效。 0: 偶校验; 1: 奇校验。
位8	PEIE: PE中断使能 (PE interrupt enable) 该位由软件设置或清除。 0: 禁止产生中断; 1: 当USART_SR中的PE为'1'时, 产生USART中断。
位7	TXEIE: 发送缓冲区空中断使能 (TXE interrupt enable) 该位由软件设置或清除。 0: 禁止产生中断; 1: 当USART_SR中的TXE为'1'时, 产生USART中断。
位6	TCIE: 发送完成中断使能 (Transmission complete interrupt enable) 该位由软件设置或清除。 0: 禁止产生中断; 1: 当USART_SR中的TC为'1'时, 产生USART中断。
位5	RXNEIE: 接收缓冲区非空中断使能 (RXNE interrupt enable) 该位由软件设置或清除。 0: 禁止产生中断; 1: 当USART_SR中的ORE或者RXNE为'1'时, 产生USART中断。
位4	IDLEIE: IDLE中断使能 (IDLE interrupt enable) 该位由软件设置或清除。 0: 禁止产生中断; 1: 当USART_SR中的IDLE为'1'时, 产生USART中断。
位3	TE: 发送使能 (Transmitter enable) 该位使能发送器。该位由软件设置或清除。 0: 禁止发送; 1: 使能发送。 注意: 1. 在数据传输过程中, 除了在智能卡模式下, 如果TE位上有个0脉冲(即设置为'0'之后再设置为'1'), 会在当前数据字传输完成后, 发送一个“前导符”(空闲总线)。 2. 当TE被设置后, 在真正发送开始之前, 有一个比特时间的延迟。
位2	RE: 接收使能 (Receiver enable) 该位由软件设置或清除。 0: 禁止接收; 1: 使能接收, 并开始搜寻RX引脚上的起始位。
位1	RWU: 接收唤醒 (Receiver wakeup) 该位用来决定是否把USART置于静默模式。该位由软件设置或清除。当唤醒序列到来时, 硬件也会将其清零。 0: 接收器处于正常工作模式; 1: 接收器处于静默模式。 注意: 1. 在把USART置于静默模式(设置RWU位)之前, USART要已经先接收了一个数据字节。否则在静默模式下, 不能被空闲总线检测唤醒。 2. 当配置成地址标记检测唤醒(WAKE位=1), 在RXNE位被置位时, 不能用软件修改RWU位。

设置串口寄存器使能位

串口发送单字符(使用库函数 USART_SendData)

硬件连接: PA9-Tx > 内部连接
PA10-Rx

电脑通过USB线连开发板(USB虚拟为串口使用)

程序功能:

STM32向PC发送数据, PC通过串口助手

查看

USART1 重映像

复用功能

USART1-TX

USART1-RX

无重映像

USART1-REMAP=0



USART1-REMAP=1

PB6

PB7

串口1发送端为引脚PA9, 接收端为引脚PA10

使能GPIOA时钟源: RCC-APB2PeriphClockCmd(RCC-APB2Periph-GPIOA, ENABLE);

使能串口1时钟源: RCC-APB2PeriphClockCmd(RCC-APB2Periph-USART1, ENABLE);

USART1-TX(PA9)应配置为推挽复用输出, USART1-RX(PA10)应配置为浮空输入/上拉输入.

```
void GPIO_Config(void)
```

```
{
```

```
    GPIO_InitTypeDef GPIO_InitStructure;
```

```
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
```

```
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
```

```
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;
```

```
    GPIO_Init(GPIOA, &GPIO_InitStructure);
```

```
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
```

```
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
```

```
    GPIO_Init(GPIOA, &GPIO_InitStructure);
```

```
}
```

串口初始化 (115200 - 8 - N - 1)

使能串口

```
void USART-Config(void)
```

```
{
```

```
    USART-InitTypeDef USART-InitStructure;
```

```
    USART-InitStructure.USART-BaudRate = 115200; // 设置波特率 = 115200 bps
```

```
    USART-InitStructure.USART-WordLength = USART-WordLength-8b; // 8位数据位
```

```
    USART-InitStructure.USART-StopBits = USART-StopBits-1; // 1位停止位
```

```
    USART-InitStructure.USART-Parity = USART-Parity-No; // 无校验位
```

```
    USART-InitStructure.USART-HardwareFlowControl = USART-HardwareFlowControl-None; // 不使用硬件流控
```

```
    USART-InitStructure.USART-Mode = USART-Mode-Tx | USART-Mode-Rx; // 允许发送、接收
```

```
    USART-Init(USART1, &USART-InitStructure);
```

```
    USART-Cmd(USART1, ENABLE); // 串口使能
```

```
}
```

发送数据并确保发送已完成

```
USART-SendData(USART1, 'A');
```

```
while (USART-GetFlagStatus(USART1, USART-FLAG-TC) != RESET);
```

清除发送完成标志。 (发送第一位数据前操作, 否则易造成第一个数据发不出去)

```
USART-ClearFlag(USART1, USART-FLAG-TC);
```

主程序

```
int main()
```

```
{
```

```
    RCC-Config(); // 开启时钟
```

```
    GPIO-Config(); // 配置收发引脚
```

```
    USART-Config(); // 配置串口参数, 115200 - 8 - N - 1 - N
```

```
    USART-ClearFlag(USART1, USART-FLAG-TC); // 清除发送完成标志, 为首次发送做准备
```

```
    USART-SendData(USART1, 'A'); // 通过串口发送字符 'A'
```

```
    while (USART-GetFlagStatus(USART1, USART-FLAG-TC) != RESET); // 等待发送完成
```

```
    USART-SendData(USART1, 'B');
```

```
    while (USART-GetFlagStatus(USART1, USART-FLAG-TC) != RESET);
```

```
}
```

串口发送字符串 (使用自定义 Uart Printf)

串口硬件连接: PA9-TX

PA10-RX

电脑通过USB线连开发板(USB虚拟为串口使用)

程序功能: 通过可变参数函数自定义一个类似于printf()的函数 Uart-Printf(), 实现将指定格式的数据通过UART1输出

Main 函数

```
int main()
{
    RCC-Config();
    GPIO-Config();
    USART-Config();
    USART-ClearFlag(USART1, USART-FLAG-TC);
    Uart-Printf("hello, world 哈哈\n");
    Uart-Printf("a=%d\n", 10);
}
```

RCC-Config 函数

```
void RCC-Config(void)
{
    RCC-APB2PeriphClockCmd(RCC-APB2Periph-USART1, ENABLE);
    RCC-APB2PeriphClockCmd(RCC-APB2Periph-GPIOA, ENABLE);
}
```

GPIO-Config 函数、USART-Config 函数同第一个例子

// 单字节发送函数

```
void Uart-SendByte(int data)
{
    if (data == '\n')
    {
        while(USART-GetFlagStatus(USART1, USART-FLAG-TC) == RESET);
        USART-SendData(USART1, '\r');
    }
    while(USART-GetFlagStatus(USART1, USART-FLAG-TC) == RESET);
    USART-SendData(USART1, data);
}
```

// 字符串发送函数

```
void Uart-SendString(char *pt)
{
    char *dst = pt;
    while (dst < pt + strlen(pt))
    {
        Uart-SendByte(*dst++);
    }
}
```


// 利用可变参数函数封装字符串发送函数

```
void Uart_Printf(const char *fmt, ...)
```

```
{
```

```
    va_list ap; // 定义1个指向可变参数列表的指针
```

```
    char string[50]; // 使参数列表指针 ap 指向第1个可变参数
```

```
    va_start(ap, fmt); // arg N 是最后一个固定参数, 这里就是 fmt
```

```
    vsprintf(string, fmt, ap); // 将 ap 按格式 fmt 写入字符串 string 中
```

```
    va_end(ap); // 清空参数列表, 置参数指针 ap 无效, 结束可变参数获取
```

```
    Uart_SendString(string);
```

```
}
```

可变参数函数的参数列表分为 固定参数、个数可变的可变参数 两部分

基本流程:

至少一个

"..."

开辟存储可变参数的区域 → 调用 vsprintf 将可变参数按指定格式复制到缓冲区 → 调用 uart_sendstring 将缓冲区数据打印到串口

串口发送字符串 (使用重定向 printf)

串口硬件连接同上例

程序功能: 将 printf() 函数的输出从显示器输出重定向至串口输出

① C 语言输出函数默认的设备是键盘和显示器, 需要将输出的信息重定向到外设 (USART1 端口)

② 当 C 编译器检查到与 C 库函数同名的函数时, 优先采用用户编写的函数, 实现重定向.

⇒ 编写 fputc 函数

使用微库

// 发送数据的函数

要发送的数据, 实际只使用 16 位中的低 8 位

```
void USART_SendData(USART_TypeDef * USARTx, uint16_t Data);
```

```
#include "stm32f10x.h"
```

```
#include "stm32f10x-usb.h"
```

```
#include <stdio.h>
```

```
void RCC_Config(void);
```

```
void GPIO_Config(void);
```

```
void USART_Config(void);
```

```
int fputc(int ch, FILE *f);
```

```
int main()
```

```
{
```

```
    RCC_Config();
```

```
    GPIO_Config();
```

```
    USART_Config();
```

```
    USART_ClearFlag(USART1, USART_FLAG_TC);
```

```
    printf("hello, world 哈哈\n");
```

```
    printf("网址: %s\n", "www.ahut.edu.cn");
```

```
    printf("a=%d\n", 10);
```

```
}
```

微

printf v

fprintf x fopen x

要先具有库, 才能实现将 printf 函数重定向到 USART 端口
(在 MDK 的 Code Generation 中选择 Use MicroLIB)

若仍使用标准 C 库, 需用户重新编写使用半主机模式的函数

→ 实现一个字符输出

```
int fputc(int ch, FILE *f)
```

```
{
```

```
    if(ch == '\n')
```

```
    {
```

→ 检测发送是否完成

```
        while(USART_GetFlagStatus(USART1, USART_FLAG_TC) == RESET);
```

```
        USART_SendData(USART1, '\r');
```

```
    }
```

```
    while(USART_GetFlagStatus(USART1, USART_FLAG_TC) == RESET);
```

```
    USART_SendData(USART1, ch);
```

```
    return ch;
```

```
}
```

```
FlagStatus USART_GetFlagStatus(USART_TypeDef * USARTx,  
                                uint16_t USART_FLAG);
```

返回值 USART_FLAG 说明置位 (SET) 或复位 (RESET)

中断方式接收串口收到的字符串并通过串口原样输出

串口硬件连接不变

程序功能: 开发板监测串口输入, 并将输入数据通过串口原样输出, 在PC端用串口调试助手观察开发板的串口输出.

```
int main(void)
{
    u8 t;
    u8 len;
    RCC-Config(); // 串口操作所需时钟使能
    USART-DeInit(USART1); // 复位串口
    GPIO-Config(); // 串口操作所需GPIO 端口设置
    UART-Config(9600); // 串口参数设置
    NVIC-Config(); // 响应串口接收中断所需NVIC 设置
    USART-ITConfig(USART1, USART-IT-RXNE, ENABLE); // 开启串口接收中断
    USART-Cmd(USART1, ENABLE); // 使能串口

    printf("请输入数据, 以回车键结束\r\n"); // 回车 \r 0x0d
    while(1) // 收到0x0a, \n
    {
        // bit 15: 接收完成标志; bit 14: 接收到0x0d; bit 13: 接收到的有效字节数.
        if (USART-RX-STA & 0x8000) // 只要USART-RX-STA的bit 15不为0, 表明收到以回车换行为结尾的完整数据.
        {
            len = USART-RX-STA & 0x3fff; // 得到此次接收到的数据长度(字节数)
            printf("\r\n 您发送的消息为: \r\n \r\n");
            for (t = 0; t < len; t++)
            {
                USART-SendData(USART1, USART-RX-BUF[t]); // 向串口发数据
                while (USART-GetFlagStatus(USART1, USART-FLAG-TX) != SET); // 等待发送结束
            }
            printf("\r\n\r\n"); // 插入换行
            USART-RX-STA = 0; // 接收数据长度计数值清0, 准备接收下一行数据
            printf("请输入数据, 以回车键结束\r\n");
        }
    }

    void UART-Config(u32 bound)
    {
        USART-InitTypeDef USART-InitStructure;
        USART-InitStructure.USART-BaudRate = bound;
        USART-InitStructure.USART-WordLength = USART-WordLength_8b;
        USART-InitStructure.USART-StopBits = USART-StopBits_1;
        USART-InitStructure.USART-Parity = USART-Parity_No;

        USART-InitStructure.USART-HardwareFlowControl = USART-HardwareFlowControl_None;
        USART-InitStructure.USART-Mode = USART-Mode-Rx | USART-Mode-Tx; // 收发模式

        USART-Init(USART1, &USART-InitStructure); // 初始化串口
    }

    void RCC-Config(void)
    {
        RCC-APB2PeriphClockCmd(RCC-APB2Periph-USART1 | RCC-APB2Periph-GPIOA, ENABLE);
    }

    void GPIO-Config(void) 同前
}
```


// NVIC-Config() 响应串口接收中断所需NVIC设置

void NVIC-Config(void)

{
NVIC-InitTypeDef NVIC-InitStructure;

NVIC-PriorityGroupConfig(NVIC-PriorityGroup-2); // 设置NVIC中断分组2: 2~2

NVIC-InitStructure.NVIC-IRQChannel = USART1-IRQn; // IRQ通道选择

NVIC-InitStructure.NVIC-IRQChannelPreemptionPriority = 3; // 抢占优先级3

NVIC-InitStructure.NVIC-IRQChannelSubPriority = 3; // 子优先级3

NVIC-InitStructure.NVIC-IRQChannelCmd = ENABLE; // IRQ通道使能

NVIC-Init(&NVIC-InitStructure); // 依指定参数初始化NVIC寄存器

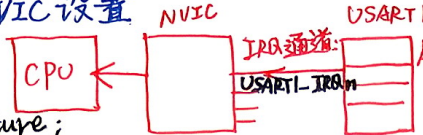
}

// 开启串口接收中断

USART-ITConfig(USART1, USART-IT-RXNE, ENABLE);

// 使能串口

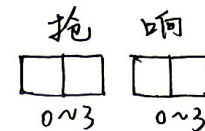
USART-Cmd(USART1, ENABLE);



0: 0-4

1: 1-3

2: 2-2



中断服务程序!

void USART1-IRQHandler(void) // 串口中断服务程序

{

us Res;

检查是否是RXNE中断发生, 是, 就做处理.

if (USART-GetITStatus(USART1, USART-IT-RXNE) != RESET) // 接收中断发生

{
状态标志: 接收缓冲器非空. (接收数据寄存器非空)

Res = USART-ReceiveData(USART1); // 读取接收到的数据(USART1 → DR)

if ((USART-RX-STA & 0x8000) == 0) // 接收未完成

{
初始状态是0

if (USART-RX-STA & 0x4000) // 接收到0x0d

接收状态

USART-RX-STA的约定: if (Res != 0x0a)

USART-RX-STA = 0; // 接收错误, 重新开始.

bit 15 - 接收完成标志

bit 14 - 接收0x0d

else
USART-RX-STA |= 0x8000; // 接收完成, 以0x0d, 0x0a为特征

bit 13 ~ 0 接收到的有效字节数

if (Res == 0x0d) // 收到0x0d

USART-RX-STA |= 0x4000; // 填进去.

else // 非结束字符填入缓冲区 USART-RX-BUF[]

{
USART-RX-BUF[USART-RX-STA & 0x3FFF] = Res;

USART-RX-STA++; // 指针下标.

if (USART-RX-STA > (USART-REC-LEN-1))

USART-RX-STA |= 0x8000 + 0x4000; // 数据行超出长度限制就截断

}

}

USART-ClearITPendingBit(USART1, USART-IT-RXNE); // 清除串口接收中断请求位, 以便响应下一次接收中断

}

串口配置的一般步骤 → 串口发送/接收引脚与GPIO引脚重合。有复用情况

① 串口时钟使能, GPIO 时钟使能: `RCC-APB2PeriphClockCmd()`;

② 串口复位: `USART_DeInit()`;

③ GPIO 端口模式设置: `GPIO_Init()`; 模式设置为 `GPIO_Mode-AF-PP`

④ 串口参数初始化: `USART_Init()`;

⑤ 开启中断并初始化NVIC: `NVIC_Init()`;
`USART_ITConfig()`;

⑥ 使能串口: `USART_Cmd()`;

⑦ 编写中断处理函数: `USARTx_IRQHandler()`;

⑧ 串口数据收发: `void USART_SendData()`;
`uint16_t USART_ReceiveData()`;

⑨ 串口传输状态获取: `FlagStatus USART_GetFlagStatus(USART_TypeDef * USARTx, uint16_t USART_FLAG);`
`void USART_ClearITPendingBit(USART_TypeDef * USARTx, uint16_t USART_IT);`